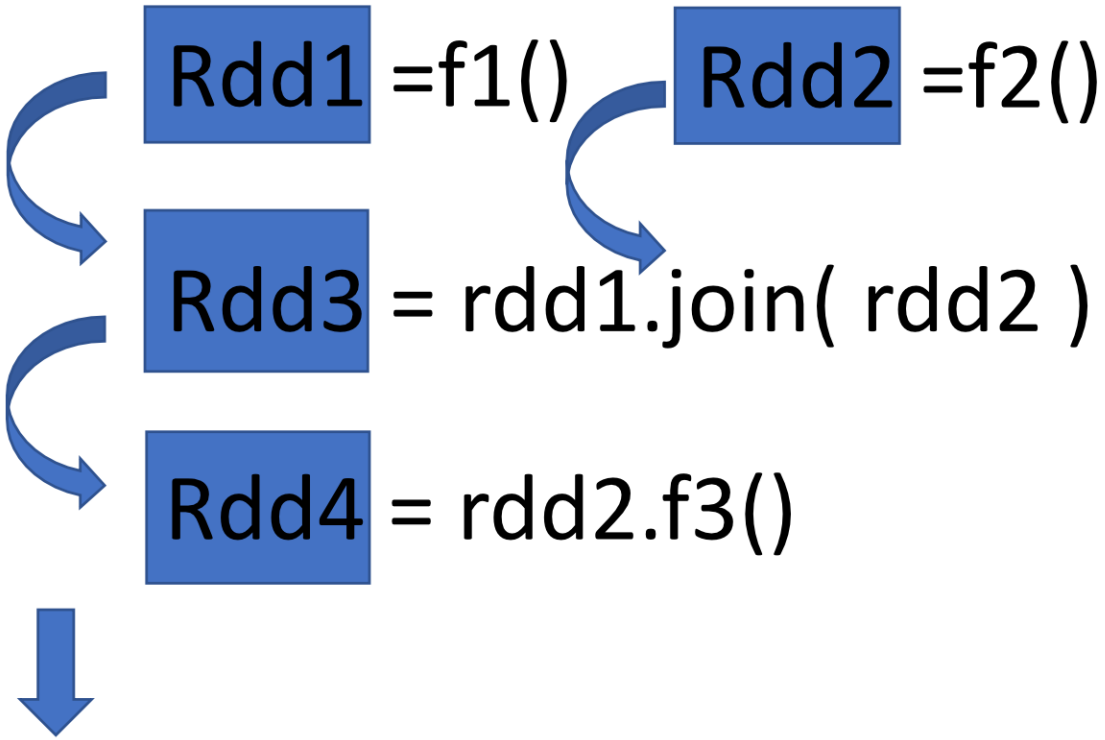
Spark Dataset Lineage Evaluation, Caching & Checkpoint

arnaud.nauwynck@gmail.com

This document:

https://github.com/arnaud-nauwynck/Presentation/big-data/dataset-lineage-eval-caching

Compose Datasets (or RDD) Transformations



Void, or List<SomethingElse>
… NO More RDD



Signature:
(SparkSession, …) -> RDD

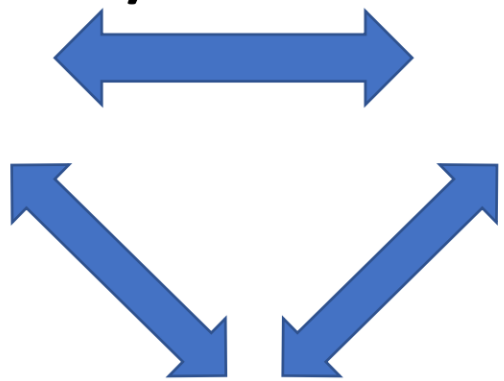
(RDD, …) -> RDD

(RDD, …) -> other

Code Declarations / Expressions / DAG

Variables Declaration
(SSA: Single Static Assignment**)**

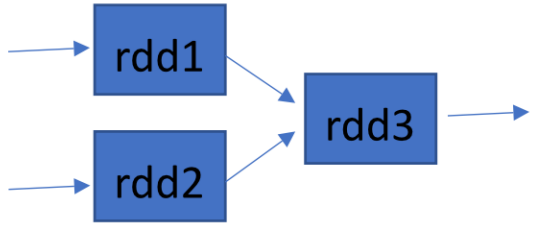
rdd1=..
rdd2=..
rdd3=..



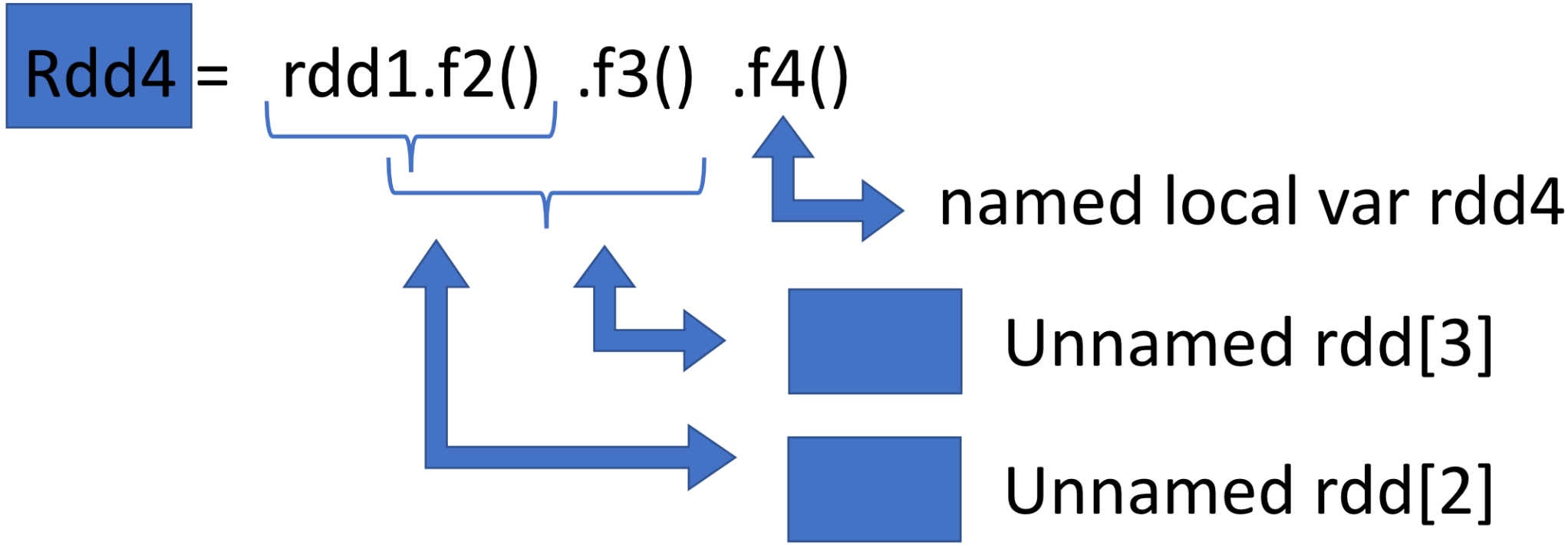
Algebraic Expressions

[f1() .join(f2())].f3()

DAG (Directed Acyclic Graph)



Intermediate (un-named) RDDs using Functional API



Lazy Transformations … then Action

Rdd1 =f1()

Defined, but lazy
=> nothing computed



Rdd2 = rdd1.f2()

=> nothing computed

Rdd3 = rdd2.f3()

=> nothing computed

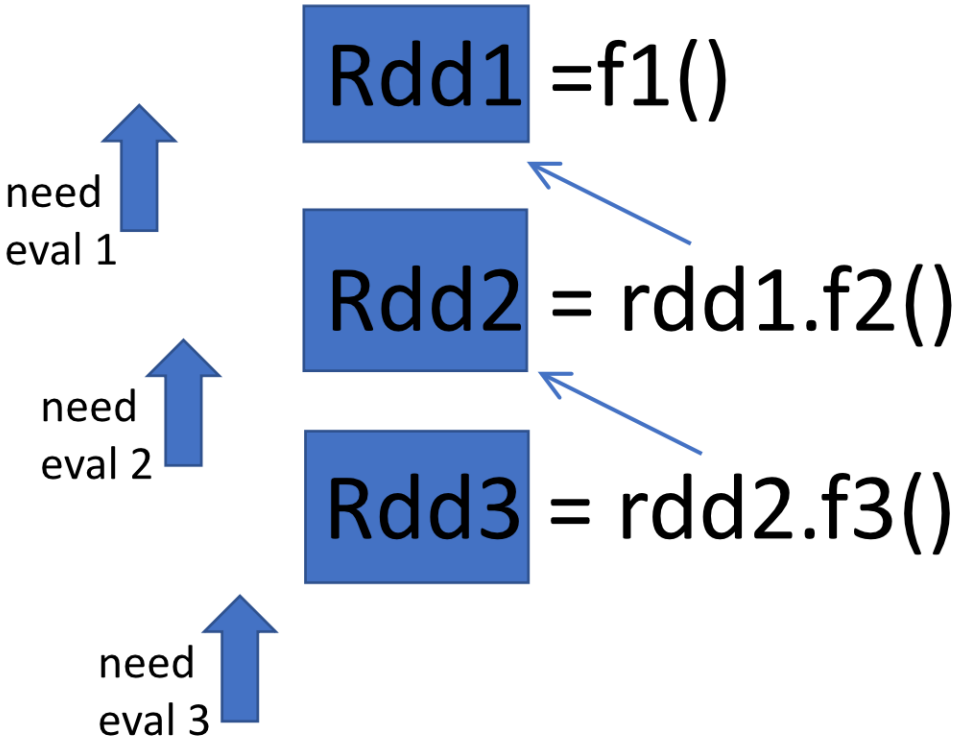


rdd3 .action();

=> Compute all : f3(f2 (f1()))



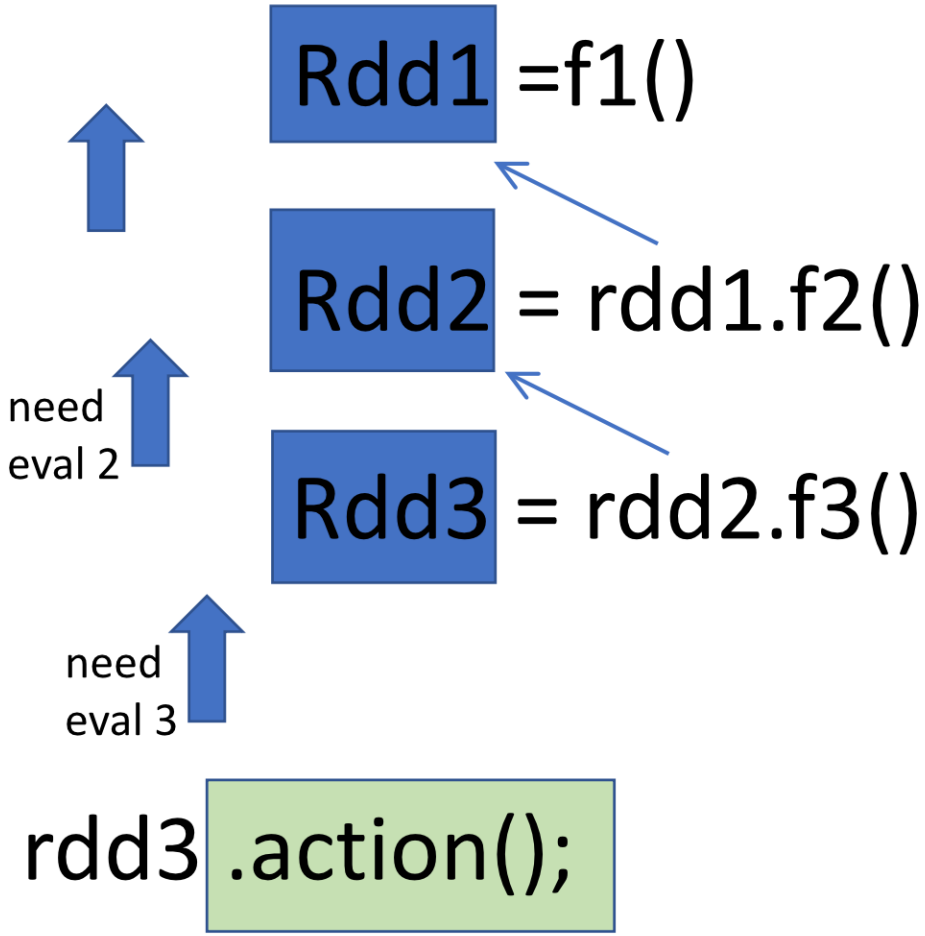
Compute Backward the Dependency Lineage

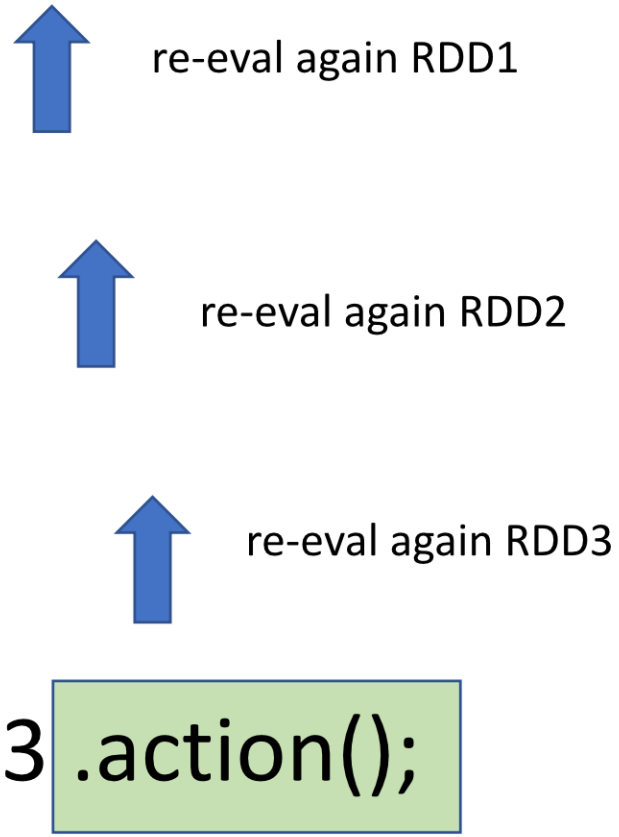


rdd3 .action(); => Compute all : f3(f2 (f1()))

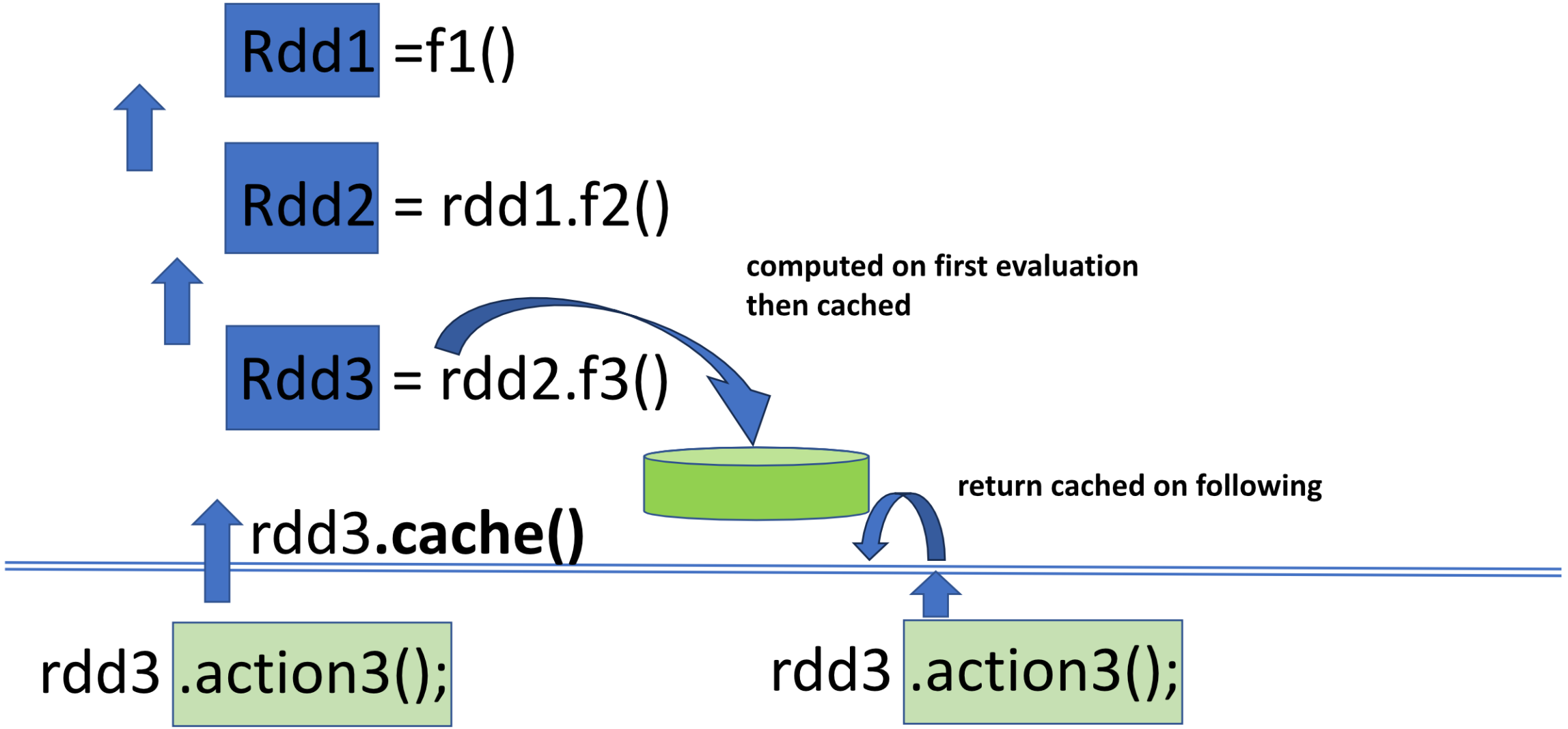


Every re-execution of an RDD
=> re-trigger Full Lineage Recomputation

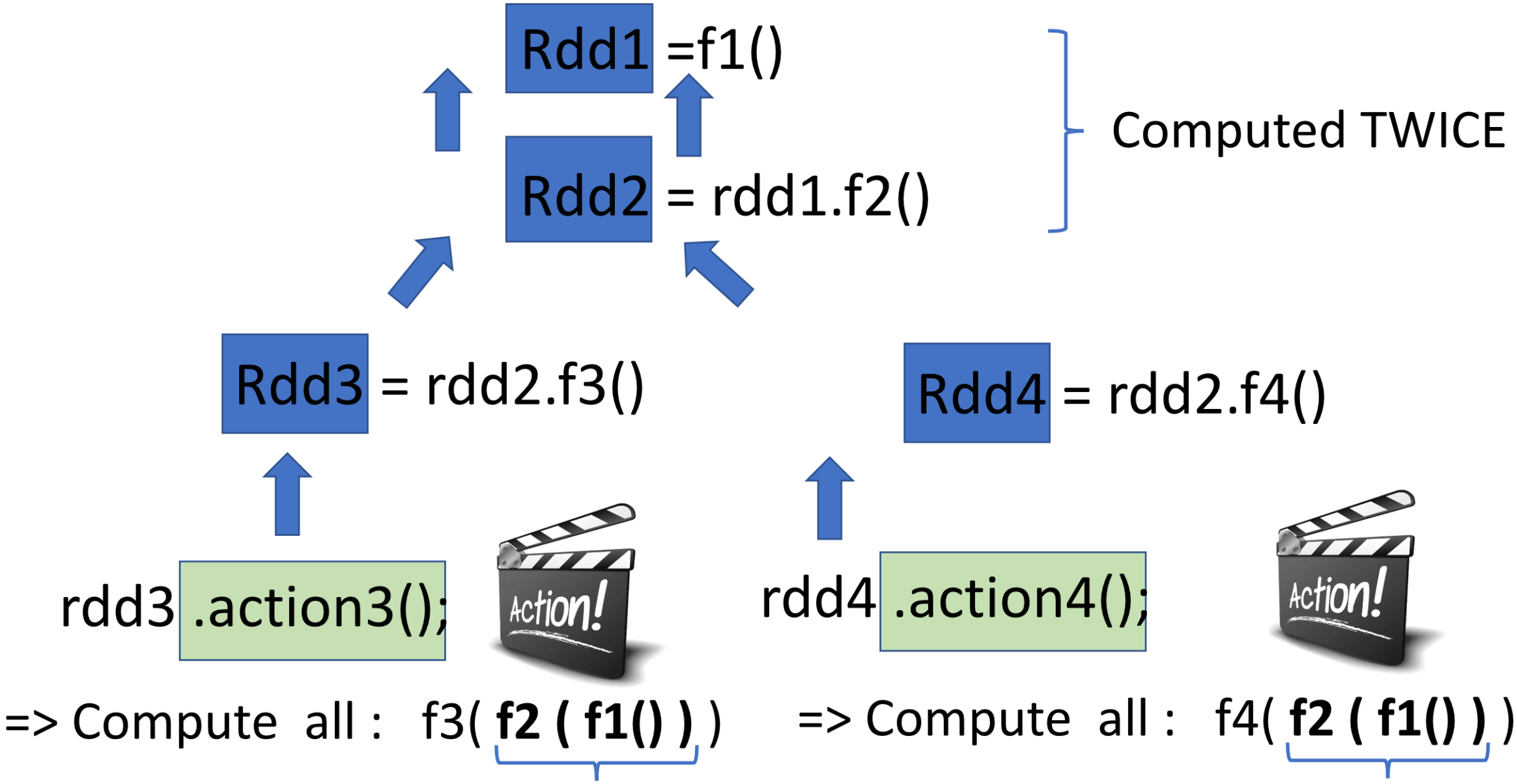




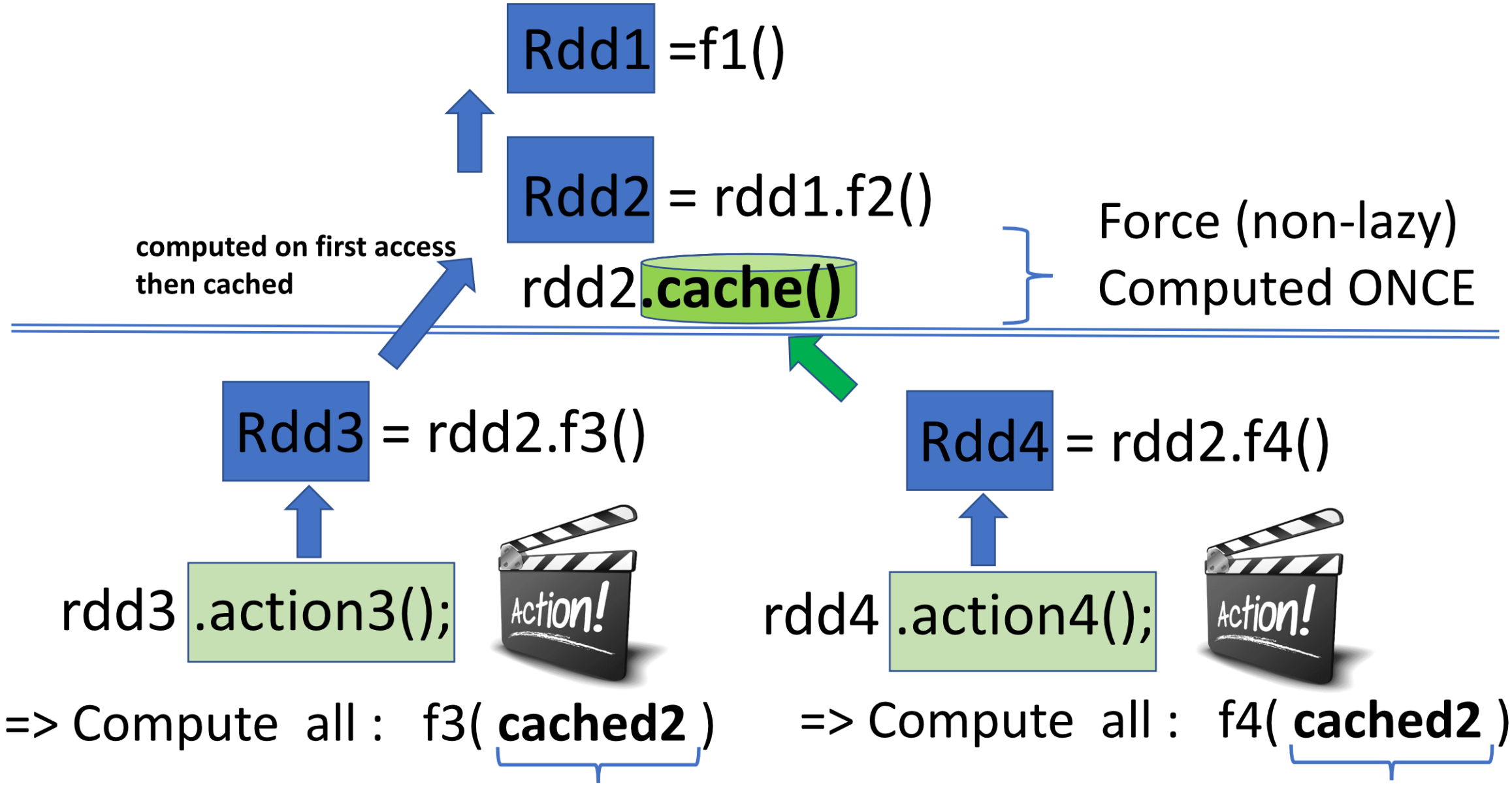
.cache() to Avoid Recomputing



RDD Shared & Lazy … not necessarily "efficient"



Cache common intermediate Result



".cache()" synonym of ".persist()" is special exception in Spark APIs

".cache()" or ".persist()" are VERY SPECIAL among all Spark APIs

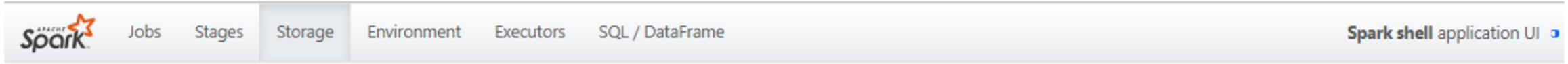
- RDD/Dataset are immutable
- APIs return a new transformed RDD/Dataset

BUT ".cache()" is THE exception :
- it modifies a field flag in the RDD
- cache() does not create or return a new object, it reuses the same

Spark Essentials: *Persistence*

transformation	*description*
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER	Store RDD as serialized Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer, but more CPU-intensive to read.
MEMORY_AND_DISK_SER	Similar to MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, **MEMORY_AND_DISK_2,** etc	Same as the levels above, but replicate each partition on two cluster nodes.

.cache() … using Memory + Disk (SWAP)



Storage

RDDs (1)

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
5	MapPartitionsRDD	Disk Memory Deserialized 1x Replicated	17	100,00%	4.2 GiB	6.1 GiB

```
WARN BlockManager: Persisting block rdd_5_13 to disk instead.
WARN MemoryStore: Not enough space to cache rdd_5_8 in memory! (computed 67.9 MiB so far)
WARN BlockManager: Persisting block rdd_5_8 to disk instead.
WARN MemoryStore: Not enough space to cache rdd_5_6 in memory! (computed 103.2 MiB so far)
WARN MemoryStore: Not enough space to cache rdd_5_11 in memory! (computed 535.3 MiB so far)
WARN BlockManager: Persisting block rdd_5_11 to disk instead.
WARN MemoryStore: Not enough space to cache rdd_5_9 in memory! (computed 540.0 MiB so far)
WARN BlockManager: Persisting block rdd_5_9 to disk instead.
WARN MemoryStore: Not enough space to cache rdd_5_10 in memory! (computed 533.1 MiB so far)
WARN BlockManager: Persisting block rdd_5_10 to disk instead.
WARN MemoryStore: Not enough space to cache rdd_5_7 in memory! (computed 237.1 MiB so far)
WARN MemoryStore: Not enough space to cache rdd_5_9 in memory! (computed 238.5 MiB so far)
WARN MemoryStore: Not enough space to cache rdd_5_12 in memory! (computed 322.2 MiB so far)
WARN MemoryStore: Not enough space to cache rdd_5_11 in memory! (computed 356.9 MiB so far)
```

Memory (and Disk) are very Limited Executor Resources

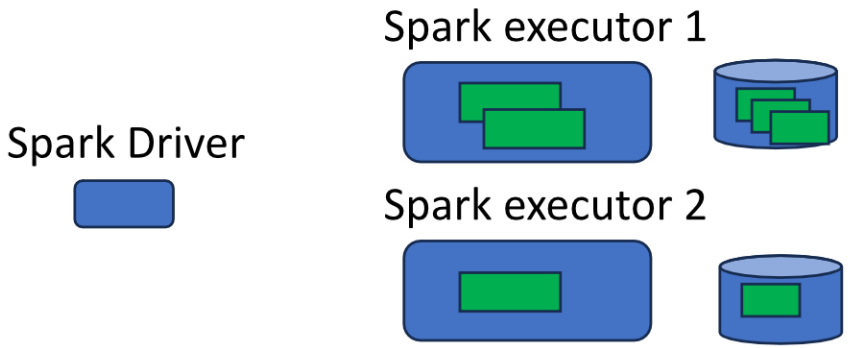
you can not cache "too much" data.
Do it only when re-using may be faster than recomputing.

when there is not enough MEMORY, it will use "DISK"

Disk may be slower than recomputing / re-fetching the source itself

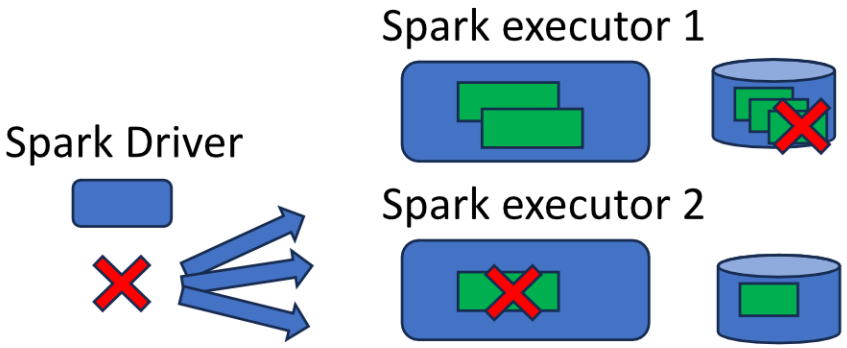
Memory and Disk are per executor (per block files)

```
{
  val ds = ... ;
  ds.cache()
  .. usage1(ds)
  .. usage2(ds)
}
```



Do you need to ".unpersist()" ? What if you don't ?

```
{
  val ds = ... ;
  ds.cache()
  .. usage1(ds)
  .. usage2(ds)
}
```



thread variable "ds" is NOT more accessible after "}"

The JVM garbage collector can reclaim "ds" memory on driver

=> spark will automatically release corresponding blocks (MEMORY / DISK) on all executors

PROBLEM: this is GC on driver-side. The driver may have plenty of free memory, so GC might not be triggered soon !
... BUT executors may be short of memory

When choosing explicitly ".unpersist()", when to do it ?

```
val ds = ....
ds.persist();
try {

    ds.action1();
    ds.action2();

} finally {
   ds.unpersist();
}

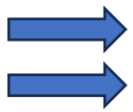
ds.action3();
```

OK
ds is cached inside try-finally code

Problem !!
ds is not cached anymore
action will re-compute FULL Lineage

Difficulty Tracking "hidden" Dependency References in Code

Looks OK
ds is cached inside try-finally code



```
Dataset transformedDs;

{
  val ds = ....
  ds.persist();
  try {

    ds.action1();
    transformedDs = ds.map(..);
    transformedDs.action();

  } finally {
    ds.unpersist();
  }
} // OK, ds not visible outside of method code
  // code would not compile
return transformedDs; // use later
```



Problem !!
transformedDs refers to ds,
which is not cached anymore
action will re-compute FULL Lineage

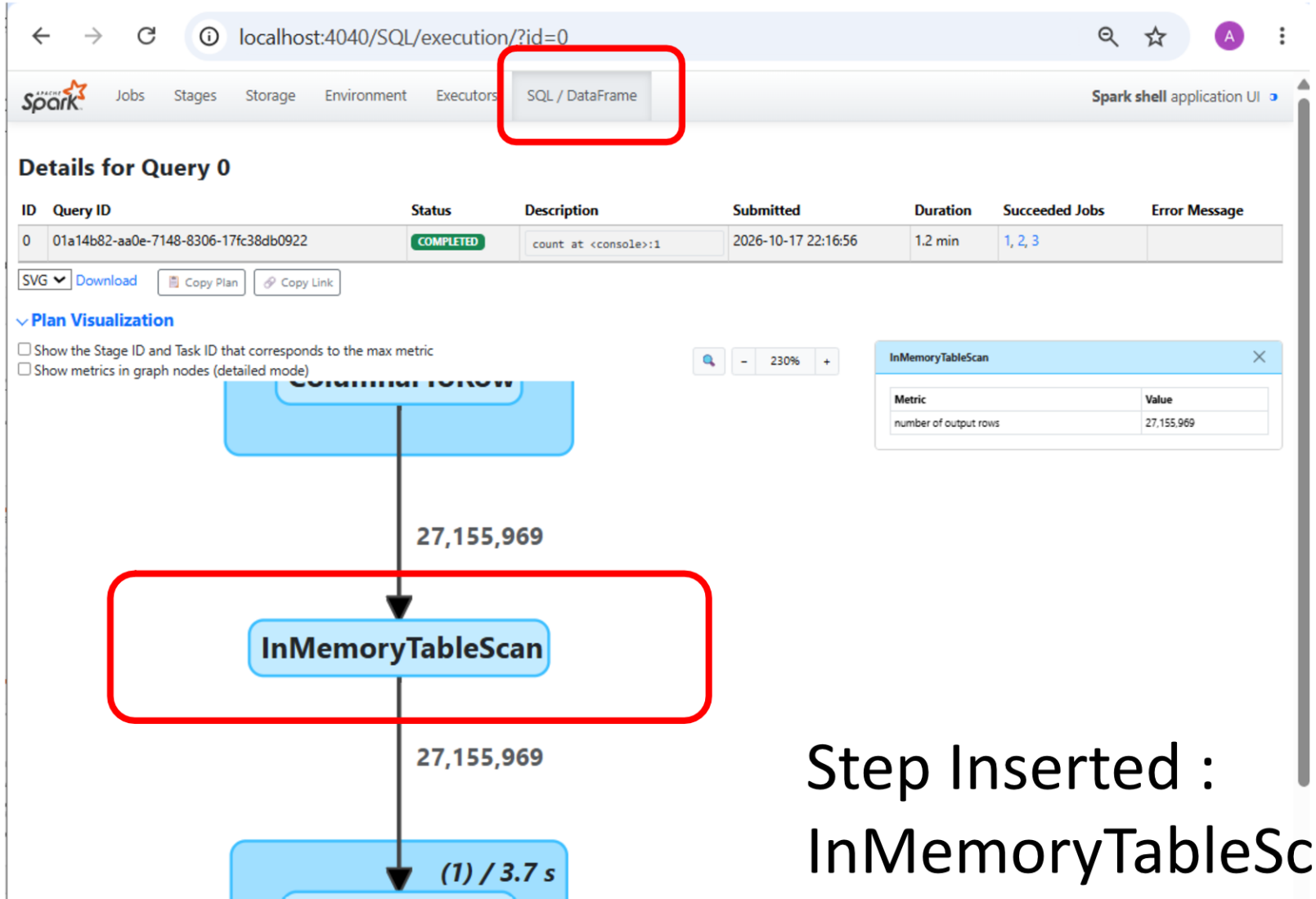
Caching- Display Result in Spark UI

when using
dataset.cache()

The **FULL DAG Lineage** is still displayed in Spark UI > Query

Some Jobs have 0 Stages, marked as "Skipped"

.cache() in SQL/DataFrame: “InMemoryTableScan”



Step Inserted : InMemoryTableScan

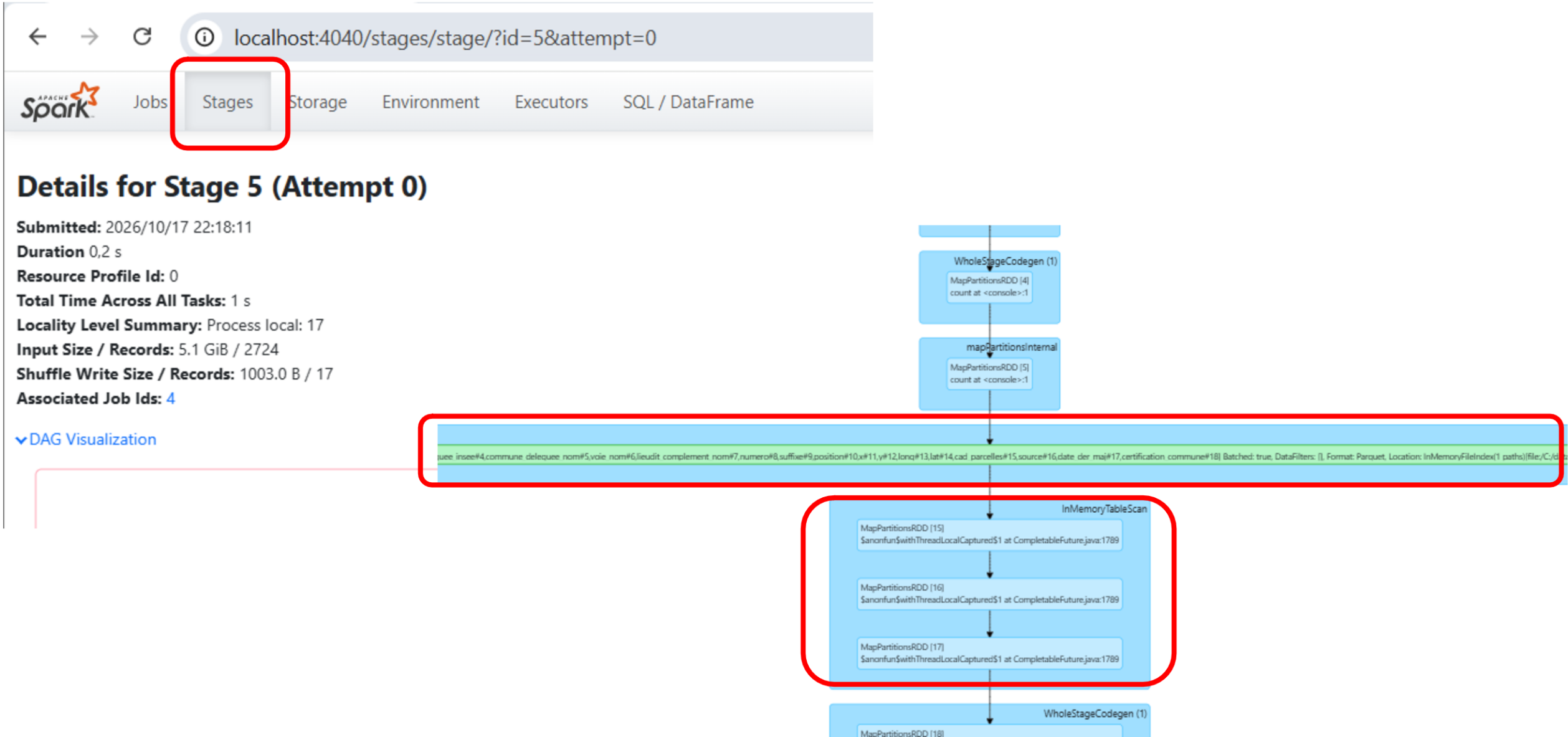
.cache() in Job



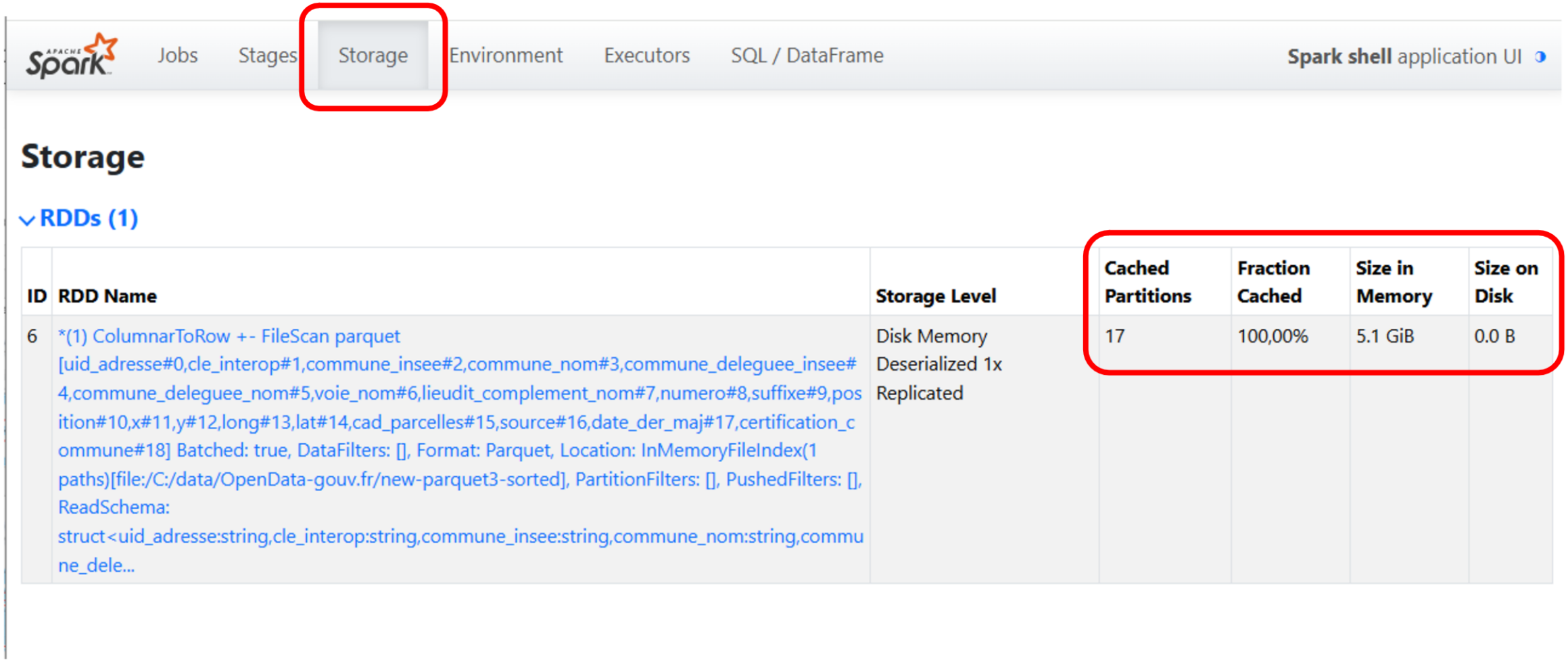
Small green point -
marker on previous step

Then InMemoryTableScan

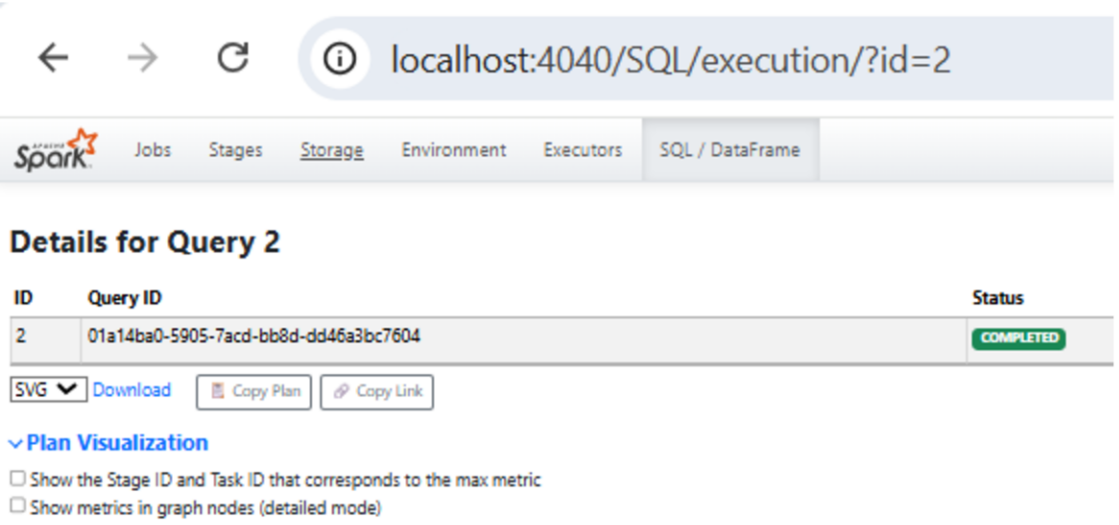
.cache() in Stage

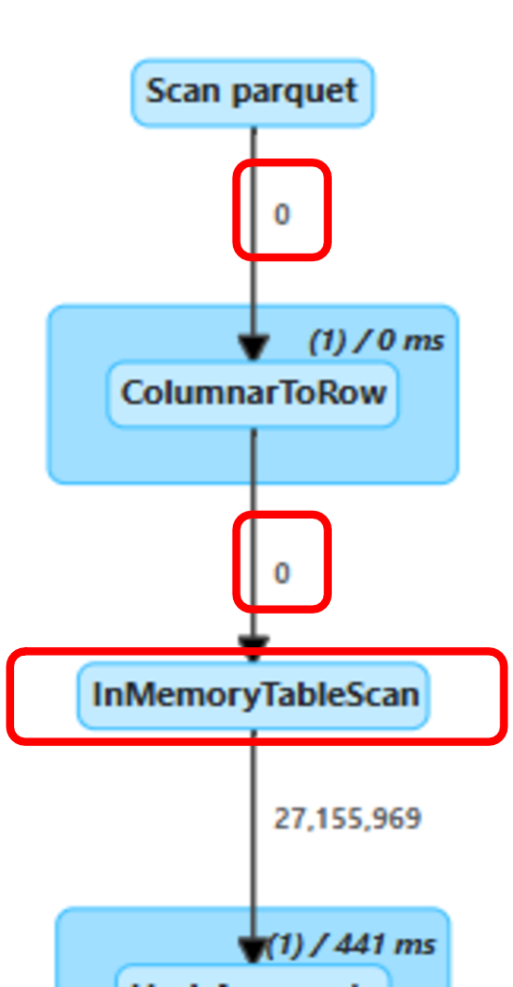


Cache Storage



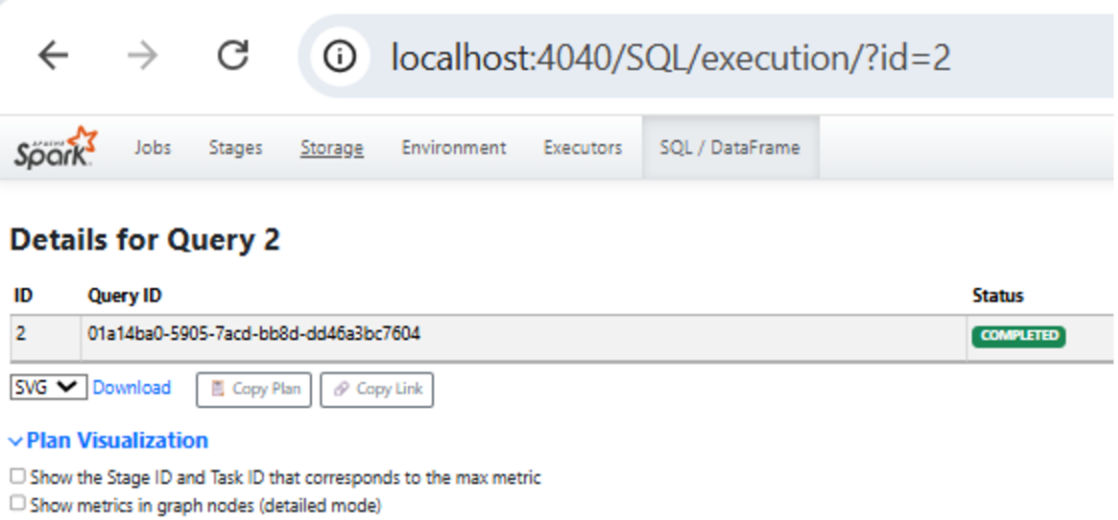
Next … Executing from cached

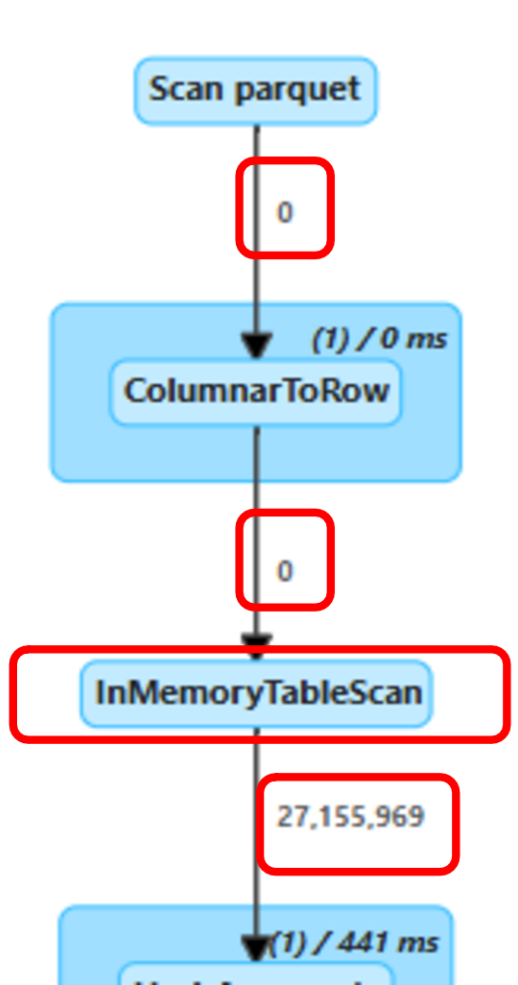




Lineage before
NOT EXECUTED, but DISPLAYED
(all steps have “0” rows)

Next … Executing from cached





Full Lineage before :
NOT EXECUTED, but DISPLAYED !
(all steps have “0” row)

“N rows” appears
After InMemoryTableScan

DAG Reading Difficulties

Full Lineage before :
NOT EXECUTED, **but DISPLAYED !**
(all steps have “0” row)

DAG Graph can be HUGE ((several screens)
…. But “size“ != “performance” / “complexity”

Visually Cutting / “Debugging” DAG

.cache() is EFFICIENT
… but still difficult to see what is consuming time

Using “**.localCheckpoint()**” or “**.checkpoint()**”
⇒ Possible to cut DAG, to simplify reading

LESS Efficient at runtime !
suitable only during Development time, for “Debugging”

Using .localCheckpoint() instead of cache()

when using

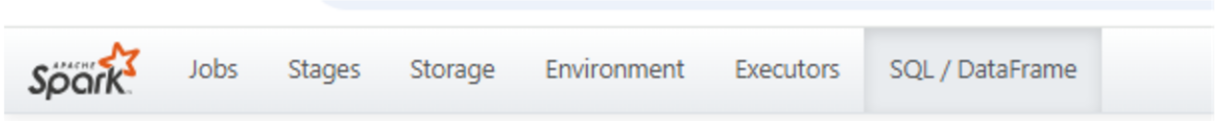
Dataset replacedDs = dataset.localCheckpoint()

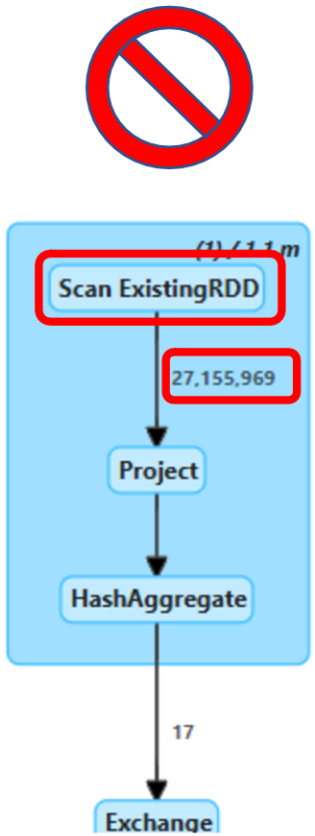
equivalent to using cached block in-memory & files

The **DAG Lineage is CUT** in Spark UI > Query.
the source is displayed as "InMemoryRDD"

=> much easier to read (even if not named/labelled correctly)

.localCheckpoint()





Lineage CUT before !!!

Starting at
“Scan Existing RDD”

Using .checkpoint()

when using

Dataset replacedDs = dataset.checkpoint()

equivalent to using saved files in persistent Hadoop storage

The **DAG Lineage is CUT** in Spark UI > Query.
the source is displayed as "FileScanRDD"

=> much easier to read (even if not named/labelled correctly)

Questions ?